



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

Corso di Ingegneria dei Linguaggi di Programmazione



Autori:

Marco Brescia

Mat. NF22500154

Giuseppe Gambardella

Mat. NF22500153

Repository GitHub:

github.com/GiuseppeGambardella/...

ANNO ACCADEMICO 2025/2026

INDICE

1	Introduzione	1
1.1	LittleButterfly: Introduzione	1
1.2	Panoramica del Progetto	1
1.2.1	Obiettivi del Progetto	2
1.3	Definizione del Compilatore	2
1.3.1	Requisiti di Sistema	2
1.3.2	Procedura di Installazione	2
1.3.3	Strumenti Utilizzati	3
1.4	Struttura del documento	3
2	Analisi lessicale	4
2.1	Analisi Lessicale: Introduzione	4
2.2	Flex	4
2.2.1	Opzioni di Configurazione	5
2.2.2	Conclusioni	5
2.3	Token	5
2.3.1	Parole Chiave	5
2.3.2	Costanti	6
2.3.3	Operatori	6
2.3.4	Identificatori	7
2.3.5	Valori Numerici dei Token	7
3	Analisi sintattica	8
3.1	Analisi Sintattica: Introduzione	8
3.2	Bison	8
3.2.1	Opzioni di Configurazione	9
3.2.2	Integrazione C++: Il ruolo di Scanner.hpp	9
3.2.3	Conclusioni	10
3.3	Le Grammatiche del Linguaggio	10
3.3.1	Blocchi	10
3.3.2	Tipo, assegnamento e dichiarazione	11
3.3.3	Istruzioni	12
3.3.4	Funzioni	13

3.3.5	Expression	14
4	AST e Symbol Table	16
4.1	Abstract Syntax Tree	16
4.1.1	Nodi dell'AST	16
4.1.2	AST Factory	17
4.1.3	Visitor Pattern	17
4.1.4	Print Visitor	18
4.2	Symbol Table	18
4.2.1	Struttura dei Simboli	18
4.2.2	Implementazione della Symbol Table	18
4.2.3	Symbol Table Visitor	19
4.2.4	Struttura del Visitor	20
5	Analisi semantica	22
5.1	Analisi Semantica: Introduzione	22
5.2	Scoping	22
5.3	Regole semantiche	23
5.3.1	Semantic Visitor	24
6	Codice intermedio	25
6.1	MLIR: Introduzione	25
6.2	MLIRGenVisitor	26
6.2.1	Struttura e Funzionamento del MLIRGenVisitor	26
6.3	Runtime Functions	28
6.3.1	Funzioni di Output	28
6.3.2	Funzioni di Input	29
6.3.3	Funzioni di Conversione	29
6.3.4	Funzione di Concatenazione di Stringhe	29
6.3.5	Funzione di Confronto di Stringhe	30
6.4	LLVM IR	30
6.5	Processo di Lowering	30
6.5.1	Introduzione al Lowering	30
6.5.2	Fasi del Processo di Lowering	31
6.6	Esecuzione	32

CAPITOLO 1

INTRODUZIONE

1.1 LittleButterfly: Introduzione

LittleButterfly è un linguaggio di programmazione che trae ispirazione da una forma di linguaggio giocoso, noto come "farfallino", utilizzato dai bambini. Il farfallino è una variante linguistica che modifica le parole inserendo, dopo ogni vocale, la stessa vocale preceduta dalla lettera "f". Ad esempio, la parola "if" diventa "ifif", e questa trasformazione si applica a tutte le vocali all'interno delle parole. Partendo da questa intuizione, l'obiettivo di questo lavoro è stato quello di progettare un linguaggio di programmazione che riprendesse l'idea di base del farfallino, ma con uno scopo più funzionale: creare un subset del linguaggio C. Il risultato è *LittleButterfly*, un linguaggio che conserva la sintassi di C, ma introduce elementi distintivi presi dalla struttura giocosa del farfallino, per rendere il codice più giocoso e, al contempo, mantenere l'idea di un linguaggio tradizionale. Per motivi di leggibilità e chiarezza, non tutte le parole chiave di C sono state trasformate secondo il modello del farfallino. Alcune di esse sono state modificate in maniera tale da evocare l'immagine del volo di una farfalla.

Nel corso di questo capitolo, verrà presentato il linguaggio *LittleButterfly* e la struttura di tale documento.

1.2 Panoramica del Progetto

Il progetto *LittleButterfly* mira a sviluppare un linguaggio di programmazione semplificato, ispirato al linguaggio C, con una sintassi ludica. Il linguaggio si distingue per l'adozione di parole chiave "giocose", create attraverso la modifica delle vocali delle parole tradizionali. Per esempio, il comando `int` diventa `ifint`, il costrutto `while` diventa `whifilefe`, e la parola `if` diventa `ifif`. Questo approccio rende il linguaggio unico, pur mantenendo una struttura di base familiare per chi ha esperienza con C. Il progetto si propone di realizzare non solo il linguaggio, ma anche un compilatore che permetta di tradurre il codice sorgente in un eseguibile ottimizzato. Il compilatore sarà sviluppato utilizzando tecnologie come **Flex**, **Bison**, **MLIR**, e **LLVM**, con l'obiettivo di offrire un processo di compilazione efficiente e modulare, capace di ottimizzare il codice sorgente.

1.2.1 Obiettivi del Progetto

In particolare, il progetto si concentra su:

- Creazione di un linguaggio di programmazione ludico.
- Sviluppo di un compilatore che traduce il codice sorgente in un eseguibile ottimizzato.
- Utilizzo di tecnologie moderne come **MLIR** e **LLVM** per garantire ottimizzazioni avanzate durante la fase di compilazione.

1.3 Definizione del Compilatore

Il compilatore *LittleButterfly* è stato sviluppato interamente in C++. Per la gestione dell'analisi lessicale e sintattica sono stati utilizzati **Flex** e **Bison**, mentre per la generazione del codice intermedio e la traduzione finale in codice macchina, il progetto fa uso di **MLIR** e **LLVM**.

1.3.1 Requisiti di Sistema

Per compilare e utilizzare il progetto, è necessario un ambiente di sviluppo C++ moderno. I requisiti principali sono i seguenti:

- **Compilatore C++:** È necessario un compilatore compatibile con lo standard C++20.
- **Sistema di Build (CMake):** Richiesta la versione 3.14 o superiore per la gestione della configurazione del progetto.
- **Framework LLVM e MLIR:** È indispensabile un'installazione completa, comprensiva delle librerie di sviluppo necessarie per la generazione e manipolazione del codice intermedio.
- **Compilazione del Progetto su Windows (MSVC):** Per costruire l'eseguibile del compilatore stesso, è stato utilizzato **MSVC** (Microsoft Visual C++). Questa scelta è necessaria per garantire la compatibilità ABI con le librerie precompilate di **MLIR** e **LLVM** su Windows.
- **Backend e Runtime (Clang):** È richiesta l'installazione di **Clang** nel sistema (accessibile da terminale). Una volta che *LittleButterfly* ha generato il codice intermedio, invoca automaticamente Clang come processo esterno per linkarlo al runtime e produrre l'eseguibile finale.

Questi strumenti sono essenziali per garantire la corretta compilazione e esecuzione del progetto. È importante che tutte le dipendenze siano correttamente configurate per evitare problemi durante la fase di build.

1.3.2 Procedura di Installazione

Per l'installazione delle dipendenze, è possibile seguire le guide ufficiali di **LLVM** e **MLIR** per il sistema operativo in uso. Una volta configurato l'ambiente, è possibile compilare il progetto utilizzando CMake e il compilatore C++.

1.3.3 Strumenti Utilizzati

- **Flex:** Utilizzato per la gestione dell'analisi lessicale, genera il codice per il lexer che suddivide l'input in token.
- **Bison:** Utilizzato per l'analisi sintattica, genera il parser che costruisce l'Abstract Syntax Tree (AST) a partire dai token forniti dal lexer.
- **MLIR:** Utilizzato per la rappresentazione intermedia del codice, che permette ottimizzazioni e trasformazioni prima della generazione del codice finale.
- **LLVM:** Utilizzato per la generazione del codice finale, compresa la fase di ottimizzazione e la creazione dell'eseguibile.

1.4 Struttura del documento

Il presente elaborato è organizzato con l'obiettivo di illustrare in modo chiaro e sistematico il progetto *LittleButterfly*. Il documento è strutturato come segue:

- **Capitolo 1: Introduzione.** Viene introdotto il progetto *LittleButterfly*, fornendone una panoramica generale e descrivendone le motivazioni e gli obiettivi principali.
- **Capitolo 2: Analisi lessicale.** In questo capitolo viene descritta la fase di analisi lessicale del compilatore. Sono presentate le scelte di configurazione di *Flex* e l'insieme dei token definiti per il linguaggio.
- **Capitolo 3: Analisi sintattica.** Il capitolo illustra l'analisi sintattica, descrivendo l'utilizzo congiunto di *Flex* e *Bison* e presentando le grammatiche adottate per il linguaggio.
- **Capitolo 4: AST e Symbol Table.** Viene descritta la struttura dell'Abstract Syntax Tree (AST) e il processo di costruzione dello stesso. Inoltre, viene discussa l'implementazione della Symbol Table e il suo ruolo nel compilatore.
- **Capitolo 5: Analisi semantica.** In questo capitolo vengono presentate le regole semantiche del linguaggio *LittleButterfly*, con particolare attenzione alla gestione dello scoping e ai controlli semantici effettuati.
- **Capitolo 6: Codice intermedio.** Il capitolo finale descrive la generazione del codice intermedio in *MLIR* e il successivo processo di lowering verso *LLVM IR*.

CAPITOLO 2

ANALISI LESSICALE

2.1 Analisi Lessicale: Introduzione

In questo capitolo verrà trattata l'analisi lessicale del linguaggio *LittleButterfly*. In particolare, verranno esplorati i seguenti aspetti:

- **Strumenti utilizzati:** sarà illustrato l'uso di **Flex** per l'implementazione dell'analizzatore lessicale, che permette di generare un analizzatore in grado di riconoscere i token nel codice sorgente.
- **Token:** saranno presentati i principali token riconosciuti dall'analizzatore, descrivendo la loro funzione e il loro ruolo nel processo di compilazione.

Questa sezione fornirà una panoramica dettagliata di come i token vengono identificati e trattati dal compilatore *LittleButterfly*.

2.2 Flex

Per l'implementazione dell'analizzatore lessicale di *LittleButterfly*, è stato utilizzato lo strumento **Flex**, un generatore di analizzatori lessicali ampiamente utilizzato. **Flex** consente di definire le regole di riconoscimento dei token tramite espressioni regolari, e automaticamente genera il codice C++ necessario per riconoscere tali pattern nel testo di input.

In pratica, il processo di analisi lessicale con **Flex** si svolge in due fasi principali:

- **Definizione delle espressioni regolari:** in un file `lexer.l` di definizione, vengono specificati i pattern per i vari token del linguaggio (ad esempio, parole chiave, identificatori, operatori, ecc.).
- **Generazione dell'analizzatore:** **Flex** compila il file di definizione generando un programma C++ che, durante l'esecuzione, esegue il riconoscimento dei token a partire dal codice sorgente.

Il vantaggio principale nell'uso di **Flex** è la sua efficienza e flessibilità nell'elaborare testi complessi, oltre alla facilità di integrazione con altre fasi del processo di compilazione, come l'analisi sintattica. In *LittleButterfly*, **Flex** viene utilizzato per pre-processare il codice sorgente, separando e classificando le singole unità lessicali in modo da prepararle per l'analisi sintattica.

2.2.1 Opzioni di Configurazione

Nel progetto, sono state utilizzate diverse opzioni di **Flex** per adattare il comportamento dell'analizzatore lessicale alle necessità specifiche del compilatore *LittleButterfly*:

- `%option c++`: Specifica che l'analizzatore lessicale deve essere scritto in C++.
- `%option noyywrap`: Disabilita la funzione `ywrap()`, che normalmente gestisce l'input da più file, ma che non è necessaria in questo caso.
- `%option yylineno`: Abilita il tracciamento del numero di linea, che consente di segnalare con precisione gli errori.
- `%option yyclass="yy::Scanner"`: Imposta la classe dell'analizzatore lessicale su `yy::Scanner`, facilitando l'integrazione con altre parti del compilatore.

2.2.2 Conclusioni

L'uso di **Flex** nel progetto *LittleButterfly* ha permesso di costruire un analizzatore lessicale altamente efficiente e facilmente integrabile. La generazione automatica del lexer, insieme al tracciamento della posizione e alla gestione degli errori, ha semplificato notevolmente lo sviluppo del compilatore, migliorando la qualità del codice e la facilità di debugging.

2.3 Token

In questa sezione vengono presentati i token riconosciuti dall'analizzatore lessicale di *LittleButterfly*. I token sono suddivisi in categorie, e per ciascun token viene fornita la regex utilizzata per il riconoscimento, il nome del token e una breve descrizione.

2.3.1 Parole Chiave

Le parole chiave sono token riservati del linguaggio che non possono essere utilizzati come identificatori. Di seguito è riportato un elenco delle principali parole chiave (vedi Tabella 2.1).

Tabella 2.1: Parole chiave nel linguaggio *LittleButterfly*

Regex	Nome del Token	Descrizione
"defef"	TOKEN_FUNC	Definisce una funzione.
Continua nella pagina successiva...		

Tabella 2.1 – Continua dalla pagina precedente

Regex	Nome del Token	Descrizione
"flyback"	TOKEN_RETURN	Restituisce il controllo dalla funzione.
"flyout"	TOKEN_PRINT	Stampa un valore a schermo.
"flyin"	TOKEN_SCAN	Esegue una scansione dell'input dell'utente.
"ifif"	TOKEN_IF	Inizio di una struttura condizionale.
"efelsefe"	TOKEN_ELSE	Ramo alternativo in una struttura condizionale.
"thefen"	TOKEN_THEN	Rappresenta l'azione da eseguire in una struttura condizionale.
"whifilefe"	TOKEN_LOOP	Inizio di un ciclo.
"ifint"	TOKEN_INT	Tipo di dato intero.
"refeafal"	TOKEN_DOUBLE	Tipo di dato floating point.
"chafar"	TOKEN_CHAR	Tipo di dato carattere.
"strifing"	TOKEN_STRING	Tipo di dato stringa.
"notype"	TOKEN_VOID	Tipo di dato void (nessun tipo).
"bofoofol"	TOKEN_BOOL	Tipo di dato booleano.
"true"	TOKEN_TRUE	Valore booleano vero.
"false"	TOKEN_FALSE	Valore booleano falso.

2.3.2 Costanti

Le costanti sono valori letterali che rappresentano numeri, caratteri o stringhe (vedi Tabella 2.2).

Regex	Nome del Token	Descrizione
[0-9]+ "." [0-9]+	TOKEN_CONST_DOUBLE	Costante di tipo double (numero decimale).
[0-9]+	TOKEN_CONST_INTEGER	Costante di tipo intero (numero intero).
\'([^\'\\\\] \\\\.)*\'	TOKEN_CONST_CHAR	Costante di tipo carattere (singolo carattere tra apici singoli, gestisce caratteri di escape).
\"([^\'\\\\] \\\\.)*\"	TOKEN_CONST_STRING	Costante di tipo stringa (sequenza di caratteri tra doppi apici).

Tabella 2.2: Costanti nel linguaggio LittleButterfly

2.3.3 Operatori

Gli operatori sono simboli utilizzati per eseguire operazioni su valori (vedi Tabella 2.3).

Tabella 2.3: Operatori nel linguaggio LittleButterfly

Regex	Nome del Token	Descrizione
"=="	TOKEN_EQ	Operatore di uguaglianza.
"<>"	TOKEN_NEQ	Operatore di disuguaglianza.
">="	TOKEN_GE	Operatore maggiore o uguale.
"<="	TOKEN_LE	Operatore minore o uguale.
"or"	TOKEN_OR	Operatore logico OR.
"and"	TOKEN_AND	Operatore logico AND.
[+\-*/=><!\&?]	(Simbolo ASCII)	Operatori a singolo carattere: aritmetici (+, -, *, /), assegnazione (=), confronto (<, >), logici (!), concatenazione (&) e speciali (?).

2.3.4 Identificatori

Gli identificatori sono nomi utilizzati per variabili, funzioni e altri oggetti nel linguaggio (vedi Tabella 2.4).

Regex	Nome del Token	Descrizione
[a-zA-Z_][a-zA-Z0-9_]*	TOKEN_ID	Identificatori. Utilizzati per assegnare nomi a variabili e funzioni definite dall'utente. Devono iniziare con una lettera o un underscore.

Tabella 2.4: Identificatori nel linguaggio LittleButterfly

2.3.5 Valori Numerici dei Token

Nel compilatore *LittleButterfly*, i token vengono gestiti attraverso un enumeratore `TokenType` che definisce i vari tipi di token riconosciuti dall'analizzatore lessicale. Ogni token è associato a un valore numerico, che viene utilizzato per rappresentare in modo univoco i diversi token durante la fase di analisi lessicale. L'enumeratore `TokenType` è strutturato in diverse categorie per facilitare la gestione e l'organizzazione del codice sorgente. Ogni token è associato a un valore numerico unico, che facilita il processo di parsing. I valori numerici sono stati scelti in modo da evitare conflitti con i valori ASCII per i caratteri singoli. Ad esempio, il token `TOKEN_CONST_INTEGER` ha il valore 258, mentre il token `TOKEN_FUNC` ha il valore 279. Questa scelta consente al compilatore di riconoscere facilmente i token durante la fase di analisi sintattica, migliorando l'efficienza del processo di compilazione.

CAPITOLO 3

ANALISI SINTATTICA

3.1 Analisi Sintattica: Introduzione

In questo capitolo verrà trattata l'analisi sintattica del linguaggio LittleButterfly. In particolare, verranno esplorati i seguenti aspetti:

- **Strumenti utilizzati:** sarà illustrato l'uso di **Bison** per l'implementazione dell'analizzatore sintattico (Parser), che permette di generare un analizzatore in grado di verificare la correttezza grammaticale della sequenza di token.
- **Grammatica:** saranno presentate le regole di produzione che definiscono la struttura del linguaggio.

Questa sezione fornirà una panoramica dettagliata di come la struttura logica del programma viene ricostruita e validata dal compilatore LittleButterfly.

3.2 Bison

Per la fase di analisi sintattica, *LittleButterfly* si avvale di **GNU Bison**, un generatore di parser di utilizzo generale che converte una descrizione grammaticale context-free in un programma C++ in grado di analizzare tale grammatica. Bison lavora a stretto contatto con l'analizzatore lessicale (Flex): mentre Flex scompone l'input in token, Bison riceve questi token e verifica che la loro sequenza rispetti le regole strutturali del linguaggio, costruendo infine l'Abstract Syntax Tree (AST).

Il funzionamento di Bison si articola nei seguenti passaggi:

- **Definizione della Grammatica:** Nel file `parser.y`, vengono definite le regole di produzione del linguaggio in formato BNF (Backus-Naur Form), specificando come i token terminali si combinano per formare strutture non terminali (come espressioni, istruzioni e funzioni).
- **Generazione del Parser:** Bison compila il file di specifica generando una classe parser in C++ (basata sull'algoritmo LALR(1)) che orchestra il processo di analisi e invoca le azioni semantiche associate a ogni regola grammaticale.

3.2.1 Opzioni di Configurazione

La configurazione di Bison è stata ottimizzata per generare un parser C++ moderno, essenziale per la gestione dei puntatori intelligenti utilizzati nell'AST. Le principali direttive utilizzate includono:

- `%skeleton "lalr1.cc"`: Istruisce Bison a generare un parser in C++ standard invece del classico C, permettendo l'uso di classi e namespace.
- `%require "3.2"`: Assicura che la versione di Bison sia almeno la 3.2, necessaria per alcune funzionalità moderne del C++.
- `%define api.value.type variant`: Questa opzione fondamentale abilita l'uso di tipi `std::variant` (o simili) per i valori semantici, sostituendo la vecchia `union` del C. Ciò permette di passare oggetti complessi come `std::unique_ptr<ASTNode>` tra le regole grammaticali in modo sicuro e senza memory leak.
- `%locations`: Abilita il calcolo automatico delle posizioni (riga e colonna) per ogni token e nodo sintattico, funzionalità cruciale per generare messaggi di errore precisi.
- `%define parse.error verbose`: Configura il parser per generare messaggi di errore dettagliati (es. "unexpected TOKEN_A, expecting TOKEN_B") invece di un generico "syntax error".
- `%param { yy::Scanner& scanner }`: Definisce i parametri che il costruttore del parser accetterà. In questo caso, viene passato un riferimento all'istanza dello scanner (`yy::Scanner`), permettendo al parser di richiedere i token direttamente all'oggetto Flex incapsulato.

3.2.2 Integrazione C++: Il ruolo di Scanner.hpp

Uno degli aspetti più complessi nell'utilizzo congiunto di Flex e Bison in un ambiente C++ moderno è la gestione della comunicazione tra i due strumenti. Flex nasce originariamente per generare codice C procedurale, mentre la configurazione scelta per Bison (`lalr1.cc`) genera una classe parser orientata agli oggetti.

Per colmare questo divario architetturale, è stato introdotto il file `Scanner.hpp`, che definisce la classe `Scanner`. Questa classe funge da "ponte" (wrapper) e svolge tre funzioni fondamentali:

1. **Incapsulamento**: La classe `Scanner` eredita da `yyFlexLexer` (la classe base fornita da Flex per il C++). Questo permette di nascondere i dettagli implementativi di basso livello di Flex e di esporre un'interfaccia pulita al resto del compilatore.
2. **Override di `yylex()`**: La funzione `yylex` generata nativamente da Flex ha una firma incompatibile con quella attesa dal parser Bison moderno. In `Scanner.hpp`, il metodo viene ridefinito per accettare i tipi corretti generati da Bison (in particolare per gestire i valori semantici `variant` e le locazioni).

3. **Gestione dello Stato:** Mantiene il riferimento agli stream di input e output, permettendo al compilatore di analizzare file o stringhe in modo trasparente.

Grazie a questa astrazione, nel file `main.cpp` è possibile istanziare un oggetto **Scanner**, passarlo al costruttore del **Parser** (come definito dalla direttiva `%param`) e avviare la compilazione mantenendo una netta separazione delle responsabilità.

3.2.3 Conclusioni

L'integrazione di Bison ha permesso di definire la grammatica di *LittleButterfly* in modo dichiarativo e manutenibile. L'adozione dello scheletro C++, combinata con l'uso della classe interfaccia **Scanner**, ha reso possibile la costruzione dell'Abstract Syntax Tree direttamente durante il parsing, sfruttando la gestione automatica della memoria del C++ moderno e garantendo una robusta gestione degli errori sintattici.

3.3 Le Grammatiche del Linguaggio

In questa sezione vengono presentate le principali regole grammaticali utilizzate per definire la sintassi di *Little Butterfly*. Le regole grammaticali sono scritte in **Bison** e seguono la notazione della Backus-Naur Form (BNF). Ogni regola è descritta tramite una produzione che associa simboli non terminali a una sequenza di simboli terminali e non terminali. Le seguenti tabelle separano le regole grammaticali in categorie distinte.

3.3.1 Blocchi

Le dichiarazioni globali definiscono variabili e funzioni nel programma.

Programma

Grammatica	Descrizione
<code>program: global_declarations function_declaration</code>	Programma in <i>LittleButterfly</i> .

Tabella 3.1: Regola del programma *LittleButterfly*

Dichiarazioni globali

Tabella 3.2: Regole per le dichiarazioni globali nel linguaggio *Little Butterfly*

Grammatica	Descrizione
<code>global_declarations: empty</code>	La lista di dichiarazioni globali è vuota (caso base).
Continua nella pagina successiva...	

Tabella 3.2 – Continua dalla pagina precedente

Grammatica	Descrizione
global_declarations: global_declarations function_declaration	Regola ricorsiva: aggiunge una dichiarazione di funzione alla lista esistente.
global_declarations: global_declarations variable_declaration ;	Regola ricorsiva: aggiunge una dichiarazione di variabile (terminata da punto e virgola).

Blocco di istruzioni

Tabella 3.3: Regole per il blocco di istruzioni

Grammatica	Descrizione
instructions: <i>empty</i>	La lista di istruzioni è vuota (blocco vuoto).
instructions: instructions instruction	Regola ricorsiva: esegue la lista di istruzioni precedente seguita dalla nuova istruzione corrente.

3.3.2 Tipo, assegnamento e dichiarazione

In questa sezione tratteremo delle grammatiche dei tipi, della dichiarazione di variabili e dell'assegnamento.

Tipi

Grammatica	Descrizione
INT	Tipo intero.
DOUBLE	Tipo reale.
CHAR	Tipo carattere.
STRING	Tipo stringa.
BOOL	Tipo booleano.
VOID	Tipo void.

Tabella 3.4: Regole per la dichiarazione di funzioni nel linguaggio LittleButterfly

Dichiarazione e assegnamento

Tabella 3.5: Regole per variabili e assegnamenti nel linguaggio LittleButterfly

Grammatica	Descrizione
variable_declaration: type TOKEN_ID	Dichiarazione semplice di una variabile (senza inizializzazione).
Continua nella pagina successiva...	

Tabella 3.5 – Continua dalla pagina precedente

Grammatica	Descrizione
variable_declaration: type TOKEN_ID '=' expression	Dichiarazione di una variabile con inizializzazione contestuale.
assignment_command: TOKEN_ID '=' expression	Assegnamento: aggiorna il valore di una variabile esistente con il risultato di un'espressione.

3.3.3 Istruzioni

Le istruzioni definiscono le operazioni eseguibili nel programma, come l'assegnamento, la stampa, la lettura e la chiamata di funzioni.

Grammatica	Descrizione
instruction: variable_declaration ;	Una dichiarazione di variabile (terminata da punto e virgola).
instruction: assignment_command ;	Un comando di assegnazione di un valore a una variabile.
instruction: print_command ;	Un comando per stampare un valore a schermo.
instruction: scan_command ;	Un comando per leggere un va- lore da input.
instruction: function_call ;	Un comando per chiamare una funzione.
instruction: TOKEN_RETURN expression ;	Un comando di ritorno dalla funzione con restituzione di un valore (flyback).
instruction: TOKEN_RETURN ;	Un comando di ritorno dalla procedura senza valore (void).
instruction: if_command	Costrutto di selezione condizio- nale (if/else).
instruction: loop_command	Costrutto di iterazione (ciclo).

Tabella 3.6: Regole per le istruzioni nel linguaggio LittleButterfly

Costrutti

Tabella 3.7: Regole per le strutture di controllo nel linguaggio Little Butterfly

Grammatica	Descrizione
if_command: TOKEN_IF expression TOKEN_THEN { instructions }	Costrutto di selezione semplice (if). Esegue il blocco se l'espressione è vera.
if_command: TOKEN_IF expression TOKEN_THEN { instructions } TOKEN_ELSE { instructions }	Costrutto di selezione completa (if- else). Esegue il primo blocco se vero, il secondo se falso.
Continua nella pagina successiva...	

Tabella 3.7 – Continua dalla pagina precedente

Grammatica	Descrizione
loop_command: TOKEN_LOOP expression TOKEN_THEN { instructions }	Costrutto di iterazione (ciclo while). Ripete il blocco finché l'espressione è vera.

Funzioni di I/O

Tabella 3.8: Regole per i comandi di Input/Output

Grammatica	Descrizione
print_command: TOKEN_PRINT (expression)	Stampa il valore risultante dall'espressione a schermo (flyout).
scan_command: TOKEN_SCAN (TOKEN_ID)	Legge un valore dall'input utente e lo memorizza nella variabile indicata (flyin).

3.3.4 Funzioni

Qui verranno trattate tutte le grammatiche relative alle funzioni e quindi chiamata, definizione, parametri e argomenti.

Function Declaration

Tabella 3.9: Regole per la dichiarazione delle funzioni

Grammatica	Descrizione
function_declaration: TOKEN_FUNC TOKEN_ID (params) : type { instructions }	Dichiarazione di una funzione standard con tipo di ritorno e parametri.
function_declaration: TOKEN_MAIN () { instructions }	Dichiarazione della funzione principale del programma (entry point "fly").

Function Call

Tabella 3.10: Regole per la chiamata di funzione

Grammatica	Descrizione
function_call: TOKEN_ID (arguments)	Chiamata a funzione: esegue la funzione identificata dal nome passando gli argomenti specificati.

Params

Tabella 3.11: Regole per la definizione dei parametri formali

Grammatica	Descrizione
params: <i>empty</i>	Nessun parametro formale.
params: type TOKEN_ID	Singolo parametro (tipo e nome).
params: params , type TOKEN_ID	Regola ricorsiva per definire molteplici parametri separati da virgola.

Arguments

Tabella 3.12: Regole per la lista degli argomenti attuali

Grammatica	Descrizione
arguments: <i>empty</i>	Nessun argomento passato alla funzione.
arguments: expression	Singolo argomento (passaggio di un valore o espressione).
arguments: arguments , expression	Regola ricorsiva per passare molteplici argomenti separati da virgola.

3.3.5 Expression

Le espressioni rappresentano operazioni su valori e variabili, come somma, sottrazione e confronto.

Tabella 3.13: Regole per le espressioni nel linguaggio LittleButterfly

Grammatica	Descrizione
expression: TOKEN_CONST_INTEGER	Un valore costante intero.
expression: TOKEN_CONST_DOUBLE	Un valore costante decimale (double).
expression: TOKEN_CONST_CHAR	Un valore costante carattere.
expression: TOKEN_CONST_STRING	Un valore costante stringa.
expression: TOKEN_TRUE	Valore booleano vero.
expression: TOKEN_FALSE	Valore booleano falso.
expression: TOKEN_ID	Riferimento a una variabile tramite il suo identificatore.
expression: expression '+' expression	Somma aritmetica.
expression: expression '*' expression	Moltiplicazione.
expression: expression '-' expression	Sottrazione.
expression: expression '/' expression	Divisione.

Continua nella pagina successiva...

Tabella 3.13 – Continua dalla pagina precedente

Grammatica	Descrizione
expression: expression '&' expression	Concatenazione di stringhe o caratteri.
expression: expression TOKEN_EQ expression	Operatore di uguaglianza (==).
expression: expression TOKEN_NEQ expression	Operatore di disuguaglianza (<>).
expression: expression '<' expression	Operatore minore di.
expression: expression '>' expression	Operatore maggiore di.
expression: expression TOKEN_LE expression	Operatore minore o uguale (<=).
expression: expression TOKEN_GE expression	Operatore maggiore o uguale (>=).
expression: expression TOKEN_AND expression	Operatore logico AND.
expression: expression TOKEN_OR expression	Operatore logico OR.
expression: '(' expression ')'	Raggruppamento di espressioni (priorità).
expression: function_call	Risultato di una chiamata a funzione.
expression: '-' expression	Operatore unario "meno" (negazione numerica).
expression: '!' expression	Operatore unario "not" (negazione logica).
expression: '?' expression	Generazione di un intero casuale tra 0 e il valore dell'espressione.

CAPITOLO 4

AST E SYMBOL TABLE

4.1 Abstract Syntax Tree

In questa sezione viene descritto l'*Abstract Syntax Tree* (AST) utilizzato nel compilatore *LittleButterfly*. L'AST rappresenta la struttura sintattica astratta del programma sorgente e costituisce una rappresentazione intermedia fondamentale per le fasi successive della compilazione, come l'analisi semantica e la generazione di codice. La progettazione dell'AST è stata guidata da criteri di modularità, estendibilità e chiarezza strutturale. In particolare, si è scelto di suddividere l'implementazione in tre componenti principali:

- la definizione dei nodi dell'AST;
- una *factory* per la creazione dei nodi;
- l'uso del *Visitor Pattern* per l'attraversamento dell'albero.

4.1.1 Nodi dell'AST

Tutti i nodi derivano da una classe base comune (**ASTNode**), che fornisce le informazioni condivise, come la posizione nel codice sorgente e l'interfaccia per il pattern Visitor. Sono stati definiti i seguenti nodi:

Tabella 4.1: Nodi AST in LittleButterfly

Grammatica	Descrizione
ProgramNode	Nodo radice del programma.
BlockNode	Nodo che serve a raggruppare altri nodi.
BooleanNode	Nodo valore booleano.
CharNode	Nodo valore carattere.
StringNode	Nodo valore stringa.
NumberNode	Nodo valore intero.
RealNode	Nodo valore reale.
TypeNode	Nodo del tipo.
Continua nella pagina successiva...	

Tabella 4.1 – Continua dalla pagina precedente

Grammatica	Descrizione
VariableNode	Nodo di una variabile.
VoidNode	Nodo void.
UnaryOpNode	Nodo per gli operatori unari.
BinaryOpNode	Nodo per gli operatori binari.
FunctionCallNode	Nodo per la chiamata a funzione.
FuncDeclNode	Nodo per la dichiarazione di una funzione.
AssignmentNode	Nodo di assegnazione.
IfNode	Nodo costruito di selezione.
LoopNode	Nodo costruito di ciclo.
ReadNode	Nodo per operazione di lettura da input.
PrintNode	Nodo di stampa.
ReturnNode	Nodo per istruzione return.
VarDeclNode	Nodo per la dichiarazione di una variabile.

Questa scelta consente di modellare in modo esplicito la struttura logica del programma, separando la sintassi concreta (gestita dal parser) dalla sua rappresentazione astratta.

4.1.2 AST Factory

Per la creazione dei nodi dell'AST è stata introdotta una *AST Factory*. La factory incapsula la logica di costruzione dei nodi, fornendo un'interfaccia uniforme per l'istanziamento degli oggetti e semplificando il codice delle azioni semantiche all'interno della grammatica Bison. L'uso della factory permette di:

- centralizzare la creazione dei nodi;
- garantire una gestione coerente della memoria tramite `std::unique_ptr`;
- ridurre l'accoppiamento tra parser e implementazione concreta dei nodi.

In questo modo, il parser si limita a descrivere la struttura sintattica del linguaggio, delegando alla factory la costruzione dell'AST.

4.1.3 Visitor Pattern

Per l'attraversamento dell'AST è stato adottato il *Visitor Pattern*. Ogni nodo dell'albero implementa un metodo `accept`, che consente a un visitor di eseguire operazioni specifiche sul nodo senza modificare la struttura dell'AST.

Questa scelta progettuale consente di:

- separare la struttura dei dati dalle operazioni eseguite su di essa;
- aggiungere facilmente nuove operazioni sull'AST senza alterare i nodi esistenti;
- migliorare la manutenibilità e l'estendibilità del compilatore.

Il Visitor Pattern risulta particolarmente adatto in un contesto di compilatori, dove l'AST deve essere attraversato più volte per svolgere compiti differenti.

4.1.4 Print Visitor

Come primo utilizzo concreto del Visitor Pattern, è stato implementato un *PrintVisitor*. Questo visitor attraversa l'AST e produce una rappresentazione testuale della struttura dell'albero, utile per scopi di debugging e validazione. Il *PrintVisitor* consente di verificare:

- la correttezza della grammatica;
- la corretta costruzione dell'AST durante il parsing;
- la corrispondenza tra il codice sorgente e la sua rappresentazione astratta.

Questo strumento si è rivelato fondamentale durante lo sviluppo del compilatore, permettendo di individuare rapidamente errori sintattici o incongruenze nella costruzione dell'albero.

4.2 Symbol Table

In questa sezione viene descritta la *Symbol Table* utilizzata nel compilatore *Little-Butterfly*. La Symbol Table è una struttura dati che associa agli identificatori del programma informazioni semantiche fondamentali, come il tipo e la natura del simbolo (variabile o funzione). Nel progetto, la Symbol Table è implementata come una struttura esplicita e compatta, pensata per supportare i controlli semantici di base e per fornire informazioni utili alle fasi successive della compilazione.

4.2.1 Struttura dei Simboli

Ogni simbolo inserito nella Symbol Table è descritto tramite la struttura `SymbolInfo`. Questa struttura raccoglie tutte le informazioni necessarie per rappresentare correttamente un identificatore all'interno del linguaggio.

In particolare, ogni simbolo contiene:

- il tipo associato al simbolo, rappresentato tramite `BasicType`;
- un flag booleano che indica se il simbolo rappresenta una funzione;
- un vettore dei tipi dei parametri, utilizzato nel caso di funzioni.

Questa rappresentazione consente di distinguere in modo chiaro tra variabili e funzioni, permettendo di effettuare controlli di coerenza sul numero e sul tipo dei parametri durante le chiamate di funzione.

4.2.2 Implementazione della Symbol Table

La Symbol Table è implementata come una mappa hash basata su `std::unordered_map`, che associa il nome del simbolo (stringa) alla corrispondente struttura `SymbolInfo`. Questa scelta progettuale consente:

- accesso rapido ai simboli tramite il loro identificatore;
- semplicità di implementazione;

- buona efficienza nelle operazioni di inserimento e ricerca.

L'interfaccia della Symbol Table espone le seguenti operazioni principali:

- **define**: inserisce un nuovo simbolo nella tabella, fallendo nel caso in cui il simbolo sia già presente;
- **lookup**: ricerca un simbolo nella tabella e restituisce le informazioni associate;
- **getTable**: fornisce accesso in sola lettura alla struttura interna;
- **printTable**: stampa il contenuto della Symbol Table a scopo di debugging.

Inserimento e Ricerca dei Simboli

L'inserimento di un simbolo avviene tramite il metodo **define**. Questo metodo verifica preventivamente l'assenza di un simbolo con lo stesso nome, evitando ridefinizioni non consentite. In caso di conflitto, l'inserimento fallisce e il simbolo non viene sovrascritto. La ricerca di un simbolo avviene tramite il metodo **lookup**, che restituisce un puntatore alla struttura **SymbolInfo** associata al nome richiesto, oppure **nullptr** se il simbolo non è presente. Questo meccanismo consente di verificare facilmente l'esistenza di un identificatore e di accedere alle sue informazioni semantiche.

Gestione delle Funzioni

Nel caso delle funzioni, la Symbol Table memorizza anche la lista dei tipi dei parametri formali. Questa informazione è utilizzata per:

- distinguere le funzioni dalle variabili;
- controllare la correttezza delle chiamate di funzione;
- verificare la compatibilità dei tipi dei parametri.

La presenza esplicita dei tipi dei parametri consente di effettuare controlli semantici già nelle prime fasi del compilatore.

Supporto al Debug

A supporto dello sviluppo e del debugging, la Symbol Table fornisce un metodo di stampa (**printTable**) che visualizza il contenuto corrente della tabella. Per ogni simbolo vengono mostrati il nome, il tipo e, nel caso di funzioni, la lista dei tipi dei parametri. Questo strumento si è rivelato utile per verificare il corretto popolamento della Symbol Table durante l'attraversamento dell'AST.

4.2.3 Symbol Table Visitor

In questa sezione viene descritto il *Symbol Table Visitor*, utilizzato nel compilatore *LittleButterfly* per popolare e verificare la Symbol Table durante l'analisi semantica del programma. Il Symbol Table Visitor implementa il *Visitor Pattern* ed è responsabile dell'attraversamento dell'Abstract Syntax Tree (AST) al fine di:

- registrare le dichiarazioni di variabili e funzioni;
- verificare l'uso corretto degli identificatori;
- individuare errori semantici come ridefinizioni o utilizzi prima della dichiarazione.

4.2.4 Struttura del Visitor

Il Symbol Table Visitor mantiene un riferimento alla Symbol Table globale e all'elenco delle linee del codice sorgente. Questo consente di associare messaggi di errore semantici a una posizione precisa nel file sorgente. Il visitor utilizza inoltre uno stato interno per tracciare la presenza di errori e per collezionare i messaggi di errore prodotti durante l'analisi. L'analisi viene svolta in due fasi principali:

1. **Registrazione delle firme:** vengono analizzate le dichiarazioni globali per registrare le firme di variabili e funzioni nella Symbol Table, senza entrare nei corpi delle funzioni.
2. **Analisi dei corpi:** una seconda visita attraversa i corpi delle funzioni e del `main`, verificando l'uso corretto degli identificatori.

Questa strategia consente di supportare riferimenti a funzioni definite successivamente nel codice sorgente.

Gestione delle Funzioni

Durante la prima fase di analisi, il visitor registra le firme delle funzioni. Per ogni funzione vengono memorizzati:

- il tipo di ritorno;
- l'informazione che il simbolo rappresenta una funzione;
- la lista dei tipi dei parametri formali.

Nel caso in cui una funzione venga ridefinita, il visitor segnala un errore semantico. Successivamente, il corpo della funzione viene analizzato separatamente, verificando l'uso delle variabili e delle espressioni contenute al suo interno.

Gestione delle Variabili

Il visitor si occupa di registrare le dichiarazioni di variabili nella Symbol Table e di verificarne l'uso corretto. In particolare:

- una variabile non può essere dichiarata con tipo `void`;
- una variabile non può essere ridefinita;
- una variabile deve essere dichiarata prima di essere utilizzata o assegnata.

Ogni violazione di queste regole genera un errore semantico associato alla riga corrispondente nel codice sorgente.

Verifica delle Espressioni e delle Istruzioni

Durante l'attraversamento dell'AST, il visitor analizza tutte le istruzioni e le espressioni del programma. Le espressioni vengono visitate ricorsivamente per garantire che tutti gli identificatori coinvolti siano correttamente definiti. Questo include:

- espressioni aritmetiche e logiche;
- assegnamenti;
- chiamate di funzione;
- strutture di controllo come `if` e `loop`.

Considerazioni Progettuali

L'uso di un visitor dedicato alla Symbol Table consente di separare nettamente:

- la costruzione dell'AST;
- la gestione delle informazioni semantiche;
- il controllo di correttezza del programma.

Questa separazione migliora la manutenibilità del compilatore e rende più semplice l'estensione futura dell'analisi semantica.

CAPITOLO 5

ANALISI SEMANTICA

5.1 Analisi Semantica: Introduzione

L'analisi semantica è una fase fondamentale del processo di compilazione, che si occupa di garantire che il programma sia semanticamente corretto. A differenza dell'analisi sintattica, che verifica la corretta struttura grammaticale del programma, l'analisi semantica verifica che le operazioni siano logicamente corrette e che le variabili e le funzioni siano usate in modo coerente con il loro significato. Nel compilatore *LittleButterfly*, l'analisi semantica si concentra su alcune regole specifiche del linguaggio, che vengono verificate utilizzando il *Semantic check Visitor*.

5.2 Scoping

Nel linguaggio *LittleButterfly*, è stato scelto di adottare uno *scoping unico* per la gestione delle variabili e delle funzioni. Questo significa che l'intero programma condivide uno spazio di nomi globale, senza la necessità di gestire ambiti locali o annidati. In un sistema con *scoping unico*, tutte le variabili e le funzioni dichiarate sono visibili in tutto il programma, senza distinzione tra scope globale e locale. Non sono previsti ambiti distinti per funzioni o blocchi di codice. Le principali regole semantiche legate a questo *scoping unico* sono:

- Ogni identificatore, che si tratti di una variabile, di una funzione o di un parametro, deve essere univoco nell'intero programma. Non è consentito avere due variabili, funzioni o parametri con lo stesso nome.
- Non è possibile dichiarare una variabile con lo stesso nome di una funzione o viceversa. Se una variabile o una funzione è già stata dichiarata, il programma genererà un errore semantico in caso di ridefinizione.
- Non esiste il meccanismo di *shadowing*: non è possibile ridefinire una variabile o una funzione all'interno di un ambito, utilizzando lo stesso nome di una variabile o funzione già esistente. Le dichiarazioni devono essere uniche.

Questa scelta di progettazione semplifica l'implementazione del compilatore, eliminando la necessità di gestire variabili locali e globali, ma introduce alcune limitazioni

come la difficoltà nella gestione di grandi progetti o di programmi complessi con molti identificatori. L'adozione di un namespace globale unico rende anche più facile il tracciamento degli errori e il debugging, poiché tutte le dichiarazioni sono visibili in un unico spazio di nomi.

5.3 Regole semantiche

In questa sezione, verranno esplorate in dettaglio le regole semantiche specifiche per ogni costrutto del linguaggio.

- **Funzione fly (main):**

- È obbligatoria la presenza di una funzione principale come punto di ingresso.
- La funzione principale deve chiamarsi tassativamente `fly`.
- La funzione `fly` non deve accettare parametri.

- **Variabili:**

- Le variabili devono essere dichiarate prima del loro utilizzo.
- Non è possibile dichiarare variabili di tipo `notype`.
- L'inizializzazione contestuale richiede una corrispondenza esatta del tipo (nessuna conversione implicita).
- L'assegnamento richiede che il valore a destra sia dello stesso identico tipo della variabile a sinistra.

- **Funzioni:**

- È possibile invocare solo funzioni preventivamente dichiarate.
- Il numero di argomenti passati deve coincidere esattamente con il numero di parametri definiti.
- Il tipo di ogni argomento deve coincidere esattamente con il tipo del parametro formale.
- Non è supportata la ricorsione diretta.
- Il valore restituito tramite `return` deve coincidere con il tipo di ritorno dichiarato nella firma.
- In una funzione dichiarata con tipo di ritorno `notype` non è necessario che sia presente l'istruzione `return`.

- **Espressioni e Operatori:**

- **Aritmetici** (+, -, *, /): Richiedono operandi numerici (`ifint` o `refeafal`).
- **Logici** (AND, OR, !): Richiedono operandi booleani (`bofoofol`).
- **Relazionali** (<, >, <=, >=): Richiedono operandi numerici.

- **Uguaglianza** (`==`, `<>`): Permissa tra tipi identici o tra tipi numerici misti. Vietata con `void`.
- **Concatenazione** (`&`): Accetta qualsiasi tipo non-void e produce una stringa.
- **Random** (`?`): Accetta solo interi (`ifint`).
- **Opposto** (`-`): Accetta operando numerico.

- **Strutture di Controllo:**

- Le condizioni di `if` e `loop` (`while`) devono essere espressioni strettamente booleane.

5.3.1 Semantic Visitor

In questa sezione viene descritto il *Semantic Visitor*, uno dei componenti principali del processo di analisi semantica nel compilatore *LittleButterfly*. Il Semantic Visitor è implementato utilizzando il *Visitor Pattern*, e si occupa di attraversare l'Abstract Syntax Tree (AST) generato dal parser per eseguire il controllo semantico del programma. Durante questa fase, il visitor interagisce con la *Symbol Table*, che memorizza le informazioni sui simboli (variabili, funzioni, tipi, ecc.), e verifica che il programma rispetti le regole semantiche del linguaggio sopra definite. Il Semantic Visitor raccoglie gli errori semantici che emergono durante l'analisi. Ogni errore contiene:

- Un messaggio che descrive l'errore semantico (ad esempio, "Variabile non dichiarata", "Tipo incompatibile").
- La riga di codice in cui l'errore è stato rilevato.
- La riga di codice sorgente che ha causato l'errore.

Gli errori semantici vengono accumulati in un buffer, che può essere successivamente utilizzato per produrre messaggi di errore dettagliati durante la fase di compilazione.

Considerazioni Finali

L'adozione del *Visitor Pattern* per l'analisi semantica consente di separare la logica di traversata dell'AST dalla logica di controllo semantico. Questo approccio migliora la modularità e la manutenibilità del compilatore, permettendo di aggiungere facilmente nuove regole semantiche senza modificare la struttura dell'AST o della Symbol Table.

CAPITOLO 6

CODICE INTERMEDIO

6.1 MLIR: Introduzione

MLIR (Multi-Level Intermediate Representation) è una rappresentazione intermedia flessibile e modulare progettata per supportare un'ampia gamma di linguaggi e passaggi di compilazione. MLIR è stato adottato in vari progetti di compilatori per la sua capacità di supportare un numero elevato di rappresentazioni e ottimizzazioni in un'unica infrastruttura. Nel contesto del compilatore *LittleButterfly*, MLIR è stato scelto come rappresentazione intermedia per consentire una gestione più efficiente delle ottimizzazioni e del codice generato. L'uso di MLIR permette di separare chiaramente le fasi di compilazione e ottimizzazione, migliorando la modularità e la flessibilità del compilatore. Il principale vantaggio nell'adottare MLIR è la sua capacità di gestire diverse astrazioni del linguaggio in modo efficiente, consentendo di applicare ottimizzazioni a diversi livelli. MLIR fornisce una rappresentazione intermedia che non è legata a un singolo linguaggio o target, ma può essere utilizzata per descrivere un ampio spettro di strutture di dati e operazioni. Questo rende MLIR estremamente utile per:

- **Modularità:** Le trasformazioni e le ottimizzazioni possono essere sviluppate in modo indipendente, separando i vari livelli di rappresentazione.
- **Flessibilità:** MLIR supporta più dialetti, che consentono di gestire vari linguaggi o rappresentazioni in modo naturale.
- **Ottimizzazioni avanzate:** La struttura modulare di MLIR permette di implementare facilmente ottimizzazioni specifiche per vari target hardware o per esigenze particolari del linguaggio.

Nel compilatore *LittleButterfly*, MLIR viene utilizzato come rappresentazione intermedia dopo l'analisi semantica e prima della generazione del codice finale. In particolare, MLIR svolge un ruolo fondamentale nelle seguenti fasi:

- **Conversione dell'AST in MLIR:** Dopo che l'AST è stato costruito e sono stati applicati i controlli semantici, il compilatore converte l'AST in una rappresentazione MLIR, che può essere più facilmente ottimizzata e trasformata.

- **Ottimizzazioni intermedie:** Una volta che il codice è rappresentato in MLIR, vengono applicate varie ottimizzazioni, come la riduzione della complessità, l'inlining delle funzioni e la gestione dei loop, per migliorare l'efficienza del codice.
- **Generazione del codice finale:** Dopo aver applicato le ottimizzazioni necessarie, il codice MLIR viene convertito in una rappresentazione finale (LLVM IR)

6.2 MLIRGenVisitor

Il *MLIRGenVisitor* è una parte fondamentale del compilatore *LittleButterfly*, responsabile della generazione del codice intermedio in formato MLIR (Multi-Level Intermediate Representation). Dopo che l'analisi semantica è stata completata, il visitor trasforma l'AST (Abstract Syntax Tree) in una rappresentazione intermedia che può essere successivamente ottimizzata e tradotta in codice di basso livello per un target hardware specifico. Nel *MLIRGenVisitor*, il codice intermedio viene generato attraverso l'interazione con la *Symbol Table*, la creazione di variabili globali, la gestione delle funzioni, e la traduzione delle espressioni aritmetiche e logiche. Questo processo è modulare e consente di applicare ottimizzazioni specifiche e di produrre una rappresentazione che può essere facilmente manipolata per ulteriori trasformazioni o generazioni di codice.

6.2.1 Struttura e Funzionamento del MLIRGenVisitor

Il *MLIRGenVisitor* utilizza il *MLIRContext* e un *MLIRSymbolTable* per costruire e gestire il codice MLIR durante la compilazione. Quando il visitor attraversa l'AST, genera operazioni di basso livello per ogni nodo dell'albero, creando funzioni, variabili e altre istruzioni necessarie.

Inizializzazione

Il *MLIRGenVisitor* è inizializzato con il contesto *MLIRContext* e una *Symbol Table*, che memorizza le informazioni relative a funzioni e variabili globali. Inizialmente, il visitor crea il modulo principale utilizzando `mlir::ModuleOp` e prepara l'inserimento di nuove funzioni e variabili.

- **Crea il modulo principale:** Il modulo principale è il contenitore radice per tutte le operazioni generate. È il primo passo per l'inizializzazione del codice MLIR.
- **Dichiarazioni delle funzioni di runtime:** Funzioni di base come `print`, `read` e `random` vengono dichiarate nel modulo.
- **Inizializzazione delle variabili globali:** Ogni variabile globale presente nella *Symbol Table* viene convertita in una variabile `memref.global` di MLIR, che viene poi inserita nel modulo.

Dichiarazione delle Funzioni di Runtime

Il *MLIRGenVisitor* definisce diverse funzioni di runtime, che sono utilizzate dal linguaggio per operazioni di I/O e conversioni di tipo. Alcuni esempi includono:

- `print_int`, `print_double`, `print_char`, `print_string` per la stampa di valori.
- `read_int`, `read_double`, `read_char`, `read_string` per la lettura di valori in input.
- Funzioni per la conversione tra tipi, come `to_string_int`, `to_string_double`, ecc.

Queste funzioni sono definite come funzioni di MLIR e vengono dichiarate tramite il metodo `declareRuntimeFunctions()` nel visitor.

Gestione delle Variabili Globali

Durante la generazione del codice, il *MLIRGenVisitor* converte le variabili globali dichiarate nel programma in variabili `memref.global`. Ogni variabile globale è rappresentata da un'operazione MLIR che allo stesso tempo alloca la memoria e la rende visibile a livello globale nel codice intermedio.

- Per ogni simbolo globale (variabile), viene creato un `memref.global` che rappresenta la memoria allocata.
- Le variabili globali sono inserite nella *MLIR Symbol Table* per facilitare il recupero dei loro valori in seguito.

Generazione del Codice per le Funzioni

Il visitor si occupa della generazione delle funzioni. Ogni funzione è tradotta in un'operazione `mlir::func::FuncOp`, che definisce la firma della funzione e le sue operazioni. Le funzioni sono gestite in due fasi principali:

- **Registrazione della firma della funzione:** Quando il visitor incontra una dichiarazione di funzione, crea un'operazione `mlir::func::FuncOp` che rappresenta la funzione nel codice intermedio.
- **Generazione del corpo della funzione:** Quando viene visitato il corpo della funzione, il visitor aggiunge i dettagli del corpo, generando le operazioni appropriate, come chiamate a funzioni, assegnamenti e ritorni.

In particolare, il visitor si assicura che la funzione principale, `fly`, venga generata correttamente, poiché rappresenta il punto di ingresso del programma.

Conversione dei Tipi

Il visitor utilizza il metodo `getMLIRType()` per mappare i tipi del linguaggio di alto livello (`int`, `double`, `bool`, ecc.) nei tipi MLIR corrispondenti. Questo è cruciale per garantire che le funzioni e le variabili siano dichiarate con i tipi corretti nel codice intermedio. La tabella di conversione include:

- `BasicType::INT → mlir::I32Type`
- `BasicType::DOUBLE → mlir::F64Type`
- `BasicType::BOOL → mlir::I1Type`
- `BasicType::CHAR → mlir::I8Type`
- `BasicType::STRING → mlir::MemRefType::get({kDynamic}, i8)`
- `BasicType::VOID → mlir::NoneType`

Questa funzione consente di creare dichiarazioni coerenti in MLIR per qualsiasi tipo di variabile o parametro.

Gestione delle Espressioni

Il visitor gestisce le operazioni aritmetiche, logiche e relazionali, traducendo ogni espressione in operazioni appropriate di MLIR. Ad esempio, operazioni di somma, sottrazione, moltiplicazione e divisione vengono tradotte in operazioni di MLIR come `arith::AddFOp`, `arith::SubIOp`, ecc. Le espressioni logiche, come `AND` e `OR`, vengono gestite come operazioni `arith::AndIOp` e `arith::OrIOp`, rispettivamente. Le espressioni vengono verificate e tradotte in una sequenza di operazioni MLIR, con i risultati memorizzati come `mlir::Value`.

6.3 Runtime Functions

Le *runtime functions* sono un insieme di funzioni predefinite che vengono utilizzate dal linguaggio *LittleButterfly* per operazioni di input/output, manipolazione di stringhe e altre operazioni utili durante l'esecuzione del programma. Queste funzioni sono implementate in C++ e sono disponibili per il programma compilato. In particolare, sono utilizzate per le operazioni di lettura e scrittura, la gestione delle stringhe e la generazione di numeri casuali.

6.3.1 Funzioni di Output

Le funzioni di output consentono al programma di scrivere su standard output. Le seguenti funzioni sono implementate per supportare la stampa di vari tipi di dati:

- `print_int(int32_t n)`: Stampa un numero intero `int32_t` seguito da una nuova riga.
- `print_double(double d)`: Stampa un numero a virgola mobile `double` seguito da una nuova riga.
- `print_char(int8_t c)`: Stampa un carattere `int8_t` seguito da una nuova riga.
- `print_string(char* allocated, char* aligned, int64_t offset, int64_t size, int64_t stride)`: Stampa una stringa utilizzando la rappresentazione `MemRef1D`. La funzione prende un puntatore alla memoria allocata e altri parametri relativi alla gestione della stringa.

Queste funzioni sono utilizzate dal linguaggio per l'output dei dati durante l'esecuzione del programma, garantendo una corretta formattazione dei dati.

6.3.2 Funzioni di Input

Le funzioni di input consentono al programma di leggere valori dallo standard input. Le seguenti funzioni sono implementate per supportare la lettura dei vari tipi di dati:

- `read_int()` : Legge un valore intero `int32_t` dallo standard input.
- `read_double()` : Legge un valore a virgola mobile `double` dallo standard input.
- `read_char()` : Legge un carattere `int8_t` dallo standard input.
- `read_string()` : Legge una stringa dallo standard input, rimuovendo eventuali caratteri di nuova linea (`\n`) e restituendo un oggetto `MemRef1D`.

Queste funzioni consentono al programma di acquisire input dall'utente in tempo reale durante l'esecuzione.

6.3.3 Funzioni di Conversione

Le funzioni di conversione permettono di trasformare vari tipi di dati in stringhe. Queste funzioni sono particolarmente utili quando è necessario stampare valori o concatenare stringhe. Le seguenti funzioni di conversione sono disponibili:

- `to_string_int(int32_t x)`: Converte un intero `int32_t` in una stringa.
- `to_string_double(double x)`: Converte un numero a virgola mobile `double` in una stringa.
- `to_string_bool(bool b)`: Converte un valore booleano in una stringa ("true" o "false").
- `to_string_char(int8_t c)`: Converte un carattere `int8_t` in una stringa.
- `to_string_string(char* allocated, char* aligned, int64_t offset, int64_t size, int64_t stride)`: Converte una stringa in una rappresentazione di tipo `MemRef1D`.

Queste funzioni sono utilizzate per generare le stringhe da stampare e per la gestione dei dati di tipo stringa all'interno del programma.

6.3.4 Funzione di Concatenazione di Stringhe

La funzione `concat_strings` consente di concatenare due stringhe. La funzione accetta due stringhe come input (ognuna rappresentata tramite `MemRef1D`) e restituisce una nuova stringa concatenata. La funzione è definita come segue:

- `concat_strings(char* a_alloc, char* a_align, int64_t a_off, int64_t a_size, int64_t a_stride, char* b_alloc, char* b_align, int64_t b_off, int64_t b_size, int64_t b_stride)`: Concatena due stringhe in una nuova stringa.

La funzione restituisce una nuova stringa rappresentata come `MemRef1D`, che contiene la concatenazione delle due stringhe originali.

6.3.5 Funzione di Confronto di Stringhe

La funzione `strcmp_strings` confronta due stringhe e restituisce un valore che indica la loro relazione:

- `strcmp_strings(char* a_alloc, char* a_align, int64_t a_off, int64_t a_size, int64_t a_stride, char* b_alloc, char* b_align, int64_t b_off, int64_t b_size, int64_t b_stride)`: Restituisce `-1` se la prima stringa è minore della seconda, `1` se è maggiore, e `0` se sono uguali.

Questa funzione è utile per confrontare le stringhe all'interno delle espressioni del programma, ad esempio per operazioni di uguaglianza.

6.4 LLVM IR

Nel compilatore *LittleButterfly*, dopo aver generato il codice intermedio in MLIR, la rappresentazione viene ulteriormente trasformata in **LLVM IR** (Intermediate Representation) attraverso un processo di *lowering*. Una volta ottenuto il codice LLVM IR, esso può essere passato a Clang per generare l'eseguibile finale. Nella prossima sezione, descriviamo come avviene la conversione da MLIR a LLVM IR, il processo di **lowering**, e infine come eseguiamo il codice risultante.

6.5 Processo di Lowering

Il processo di lowering da **MLIR** a **LLVM IR** è una fase fondamentale nel compilatore *LittleButterfly*, in cui il codice intermedio in MLIR viene trasformato in una rappresentazione di basso livello che può essere ottimizzata e compilata in codice macchina eseguibile. In questa fase, il codice viene abbassato attraverso diverse fasi di trasformazione per adattarsi al formato di LLVM IR, che è supportato dal framework LLVM per la generazione finale di codice.

6.5.1 Introduzione al Lowering

Il codice in MLIR, che è una rappresentazione intermedia di livello più alto, deve essere tradotto in **LLVM IR** per poter essere sottoposto a ottimizzazioni e generazione di codice finale. MLIR fornisce un'infrastruttura modulare e flessibile che consente di mappare le operazioni di un linguaggio di alto livello in una rappresentazione che può essere ulteriormente ottimizzata prima di generare il codice finale. Il lowering è il processo che trasforma i costrutti di linguaggi di alto livello, come variabili, funzioni e operazioni aritmetiche, nelle corrispondenti operazioni di basso livello supportate da LLVM IR.

6.5.2 Fasi del Processo di Lowering

Il processo di lowering da MLIR a LLVM IR si articola in diverse fasi, ciascuna delle quali si concentra su un aspetto specifico della traduzione. Queste fasi includono l'ottimizzazione, la conversione dei costrutti e la gestione del flusso di controllo.

1. Registrazione dei Dialetti MLIR

La prima fase del lowering consiste nel registrare i dialetti necessari all'interno del contesto MLIR. MLIR supporta vari dialetti, ognuno dei quali rappresenta un'astrazione o un'operazione specifica di un linguaggio di programmazione. Nel nostro caso, vengono registrati dialetti come:

- **BuiltinDialect**: Gestisce le operazioni di base di MLIR.
- **ArithDialect**: Gestisce le operazioni aritmetiche (somma, sottrazione, moltiplicazione, ecc.).
- **MemRefDialect**: Gestisce le operazioni relative alla memoria.
- **SCFDialect**: Gestisce le operazioni di flusso di controllo.
- **LLVMDialect**: Gestisce le operazioni per la generazione di LLVM IR.

Questi dialetti vengono registrati nel contesto di MLIR e resi disponibili per la successiva fase di abbassamento.

2. Ottimizzazione del Codice MLIR

Prima di procedere con il vero e proprio lowering, il codice MLIR viene ottimizzato per ridurre la complessità e migliorare le performance. Vengono applicati vari passaggi di ottimizzazione, tra cui:

- **CanonicalizerPass**: Esegue trasformazioni per semplificare il codice.
- **CSEPass** (Common Subexpression Elimination): Elimina le espressioni comuni.
- **LoopInvariantCodeMotionPass** (opzionale ma consigliato): Ottimizza il movimento del codice invariabile all'interno dei loop.
- **SCCPPass** (Simplify Conditional Constant Propagation): Ottimizza le espressioni condizionali.

Queste ottimizzazioni preparano il codice per la fase successiva di abbassamento, migliorando la qualità del codice finale.

3. Conversione da SCF a Flusso di Controllo

In questa fase, le operazioni di flusso di controllo (come i loop e i condizionali) vengono convertite in una rappresentazione compatibile con LLVM IR. Ad esempio, le operazioni SCF (Structured Control Flow) vengono convertite in operazioni di flusso di controllo (CF) più generiche, che sono più adatte per il backend LLVM. Questo passaggio garantisce che il codice abbia un formato compatibile con il sistema di controllo del flusso di LLVM.

4. Conversione a LLVM IR

La fase finale del lowering consiste nella traduzione del codice MLIR in un dialetto molto simile a LLVM IR. Durante questa fase vengono eseguite le seguenti conversioni:

- **Operazioni aritmetiche:** Le operazioni come somma, sottrazione, moltiplicazione e divisione vengono tradotte in operazioni LLVM IR come `add`, `sub`, `mul`, `div`.
- **Memoria:** Le operazioni di gestione della memoria, come allocazioni e deallocazioni, vengono mappate su operazioni `alloca` e `memref.global`.
- **Funzioni:** Le dichiarazioni di funzioni vengono tradotte in funzioni LLVM IR, e le chiamate alle funzioni sono mappate su istruzioni di chiamata LLVM IR.
- **Flusso di controllo:** Le istruzioni di salto condizionale (if-else) e i loop vengono tradotti in istruzioni di controllo del flusso in LLVM IR, come `br` e `condbr`.

Una volta terminate le conversioni, il modulo risultante viene tradotto in codice LLVM IR.

6.6 Esecuzione

Dopo il processo di lowering, il codice risultante in **LLVM IR** viene passato direttamente a **Clang**, che si occupa di compilarlo in un eseguibile. In questa fase, Clang esegue le seguenti operazioni:

- **Ottimizzazione del codice LLVM IR:** Clang applica le ottimizzazioni necessarie per migliorare le performance.
- **Generazione del codice macchina:** Il codice LLVM IR viene tradotto in codice macchina specifico per l'architettura di destinazione.
- **Creazione dell'eseguibile:** Clang esegue il linking con le librerie necessarie, generando il file eseguibile finale.

Una volta creato l'eseguibile, il programma può essere eseguito sulla macchina di destinazione.